LECTURE 7

Path planning

for autonomous driving

Search, robot geometry, and navigation

Introduction to Autonomous Driving · Fall 2026

Learning goals

Explain A* search and distinguish D* from D* Lite replanning.

Predict when an algorithm succeeds, slows down, or returns an unusable path.

Connect obstacle clearance and vehicle motion to global and local planning.

Planning in the autonomy stack

Perception estimates nearby objects and free space.

Localization and mapping place the robot in an environment.

Planning chooses a route and motion. Control tracks that motion.

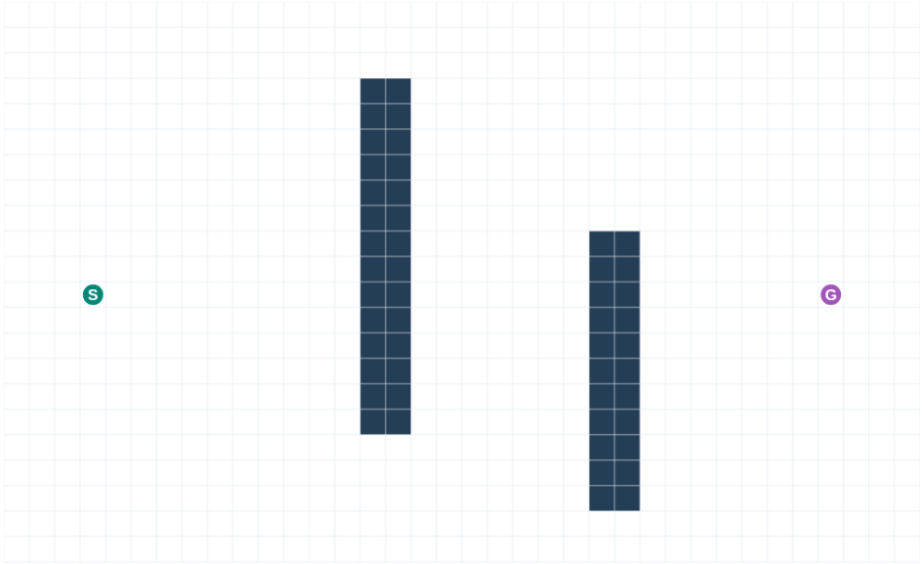
Path, trajectory, and control

Object	Describes	Example
Path	Where the robot can move	Positions and headings
Trajectory	Where and when it moves	Pose and speed over time
Control	Inputs applied now	Steering, throttle, braking

Global and local planning

Dimension	Global planning	Local planning / control
Question	Which route reaches the goal?	Which motion is feasible now?
Inputs	Map, start, destination	State, nearby objects, route
Output	Route or reference path	Short trajectory / command
Horizon	Whole task or map region	Limited distance or time
Update trigger	Goal or route/map changes	New state and observations

A map becomes a search graph



Dark navy: obstacles · Green S: start · Purple G: goal

Open interactive experiment

Nodes represent candidate robot states.

Edges represent allowed transitions.

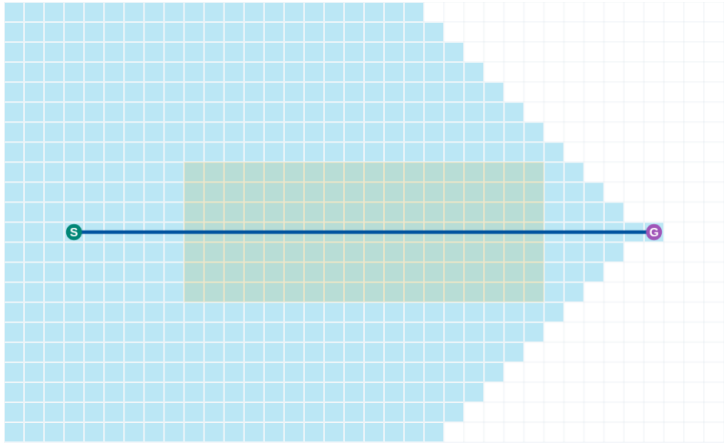
Costs express distance, terrain, risk, or time.

BFS and Dijkstra

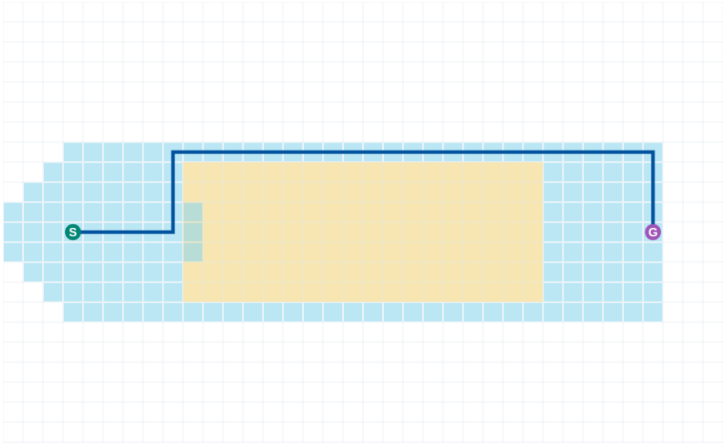
Method	Next state priority	Useful when
Breadth-first search	Fewest edges from start	All edges have equal cost
Dijkstra	Lowest accumulated cost g	Nonnegative unequal costs
Shared limitation	Search can spread widely	Large graphs consume memory

Compare BFS, Dijkstra and A*

Distance and terrain cost



BFS: shortest in number of edges



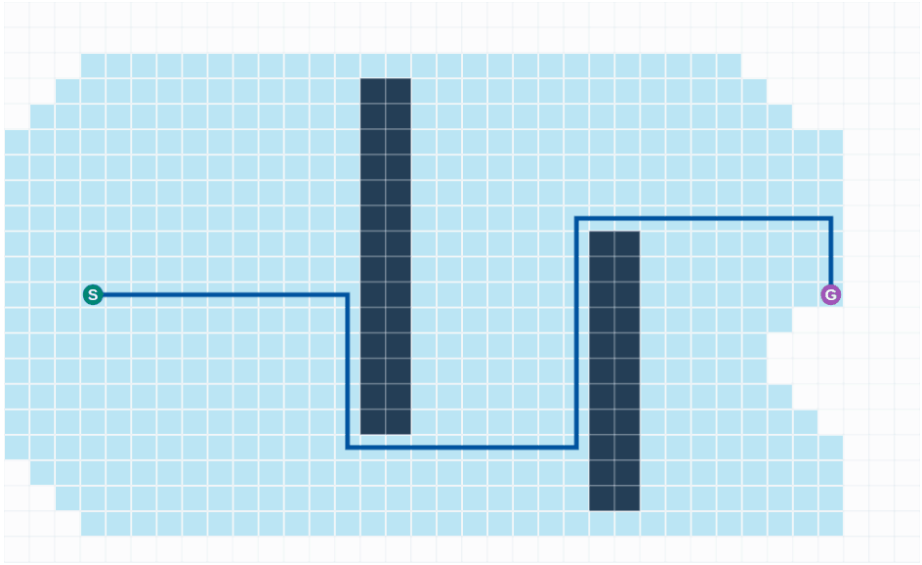
A*: lower total terrain cost

BFS crosses rough ground to use fewer steps.
A* takes a longer route with lower modeled cost.

Yellow: terrain cost 6 · Cyan overlay: expanded cells · Blue: route

Open interactive experiment

A* search



Cyan: expanded cells · Blue: final route · Dark navy: obstacles

Open interactive experiment

g measures the cost already spent.

h estimates the remaining cost.

A* expands the state with the lowest g + h.

A* priority calculation

f(n) = g(n) + h(n)

A: 6 + 11 = 17 B: 9 + 8 = 17

Both candidates have a priority of 17.

Tie breaking affects expansion order, even when the optimal cost is unchanged.

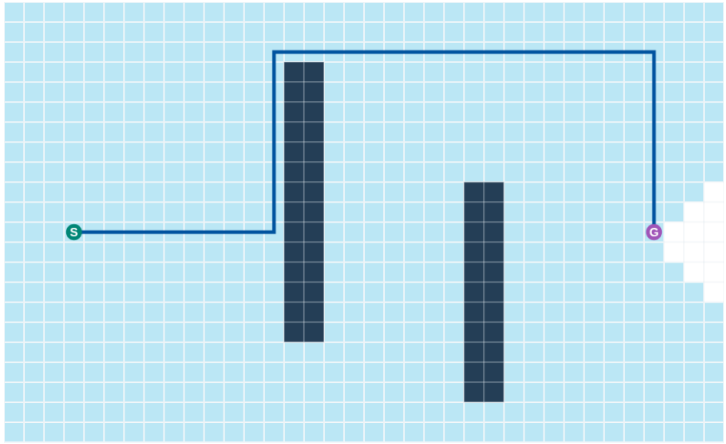
Heuristic assumptions

Graph	Lower-bound heuristic	Condition
4-neighbor grid	Manhattan distance	Unit minimum step cost
8-neighbor grid	Octile distance	Diagonal cost √2
Euclidean motion	Straight-line distance	Cost ≥ geometric length
Changed cost model	Recheck the lower bound	Match units and transitions

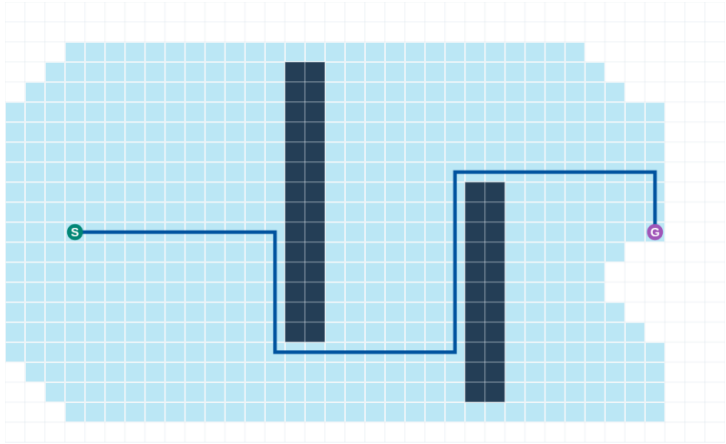
The A* search loop

1. Initialize the start with g = 0 and put it in OPEN.
2. Remove the lowest f state and relax its outgoing edges.
3. Update a neighbor when the new g improves its route.
4. Stop when the goal leaves OPEN, or OPEN becomes empty.

Dijkstra and A* on the same graph



Dijkstra: g priority



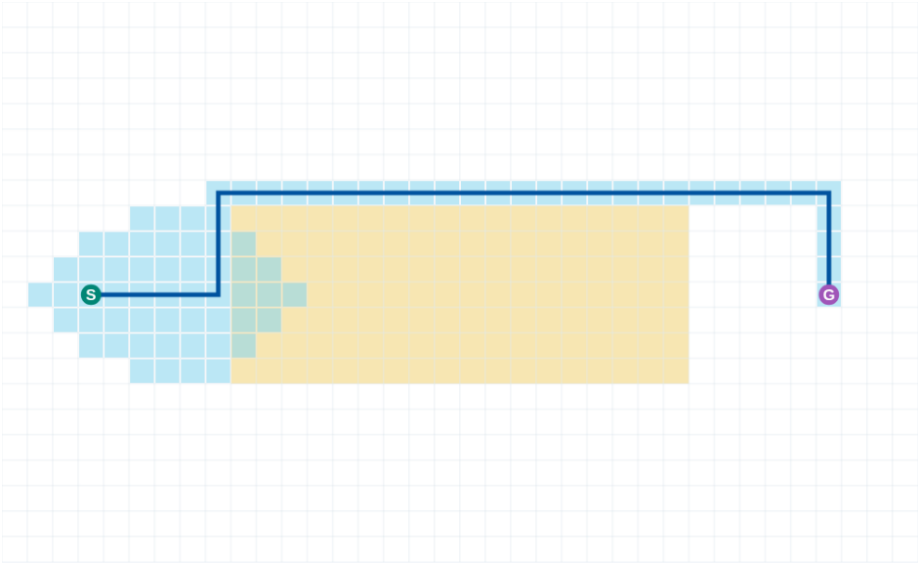
A*: g + h priority

Dijkstra spreads according to cost from the start.
A* also uses a lower bound on distance to the goal.

Cyan: expanded cells · Blue: minimum-cost route · Dark navy: obstacles

Open interactive experiment

A* and Weighted A*



Pictured: Weighted A*, w = 2 · Cyan: expanded cells · Blue: route

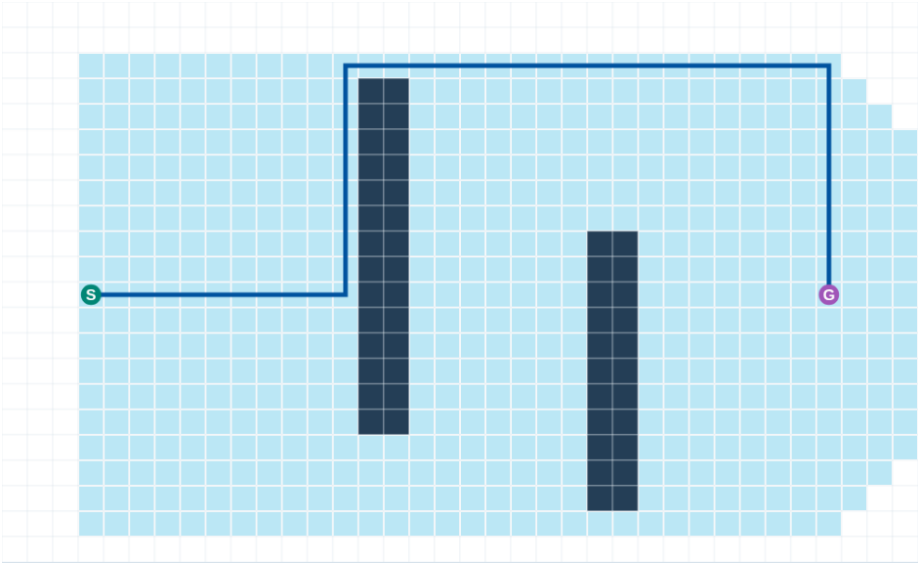
Open interactive experiment

A* minimizes graph cost with a valid heuristic and search policy.

Weighted A*: $f = g + w\,h$, with $w > 1$.

The stronger heuristic can reduce expansions but lose exact optimality.

Replanning after a map change



Cyan: initial D* Lite expansions · Blue: initial route

Open interactive experiment

A sensor discovers a blockage on the planned route.

Edge costs change when a cell becomes occupied or free.

Incremental search reuses earlier work.

D* and D* Lite

Method	Origin	Core idea
D*	Stentz, 1994	Repair routes after cost changes
D* Lite	Koenig & Likhachev, 2002	Reuse backward g / rhs values
Repeated A*	New search after each edit	Discard previous search state

D* Lite consistency

rhs(goal) = 0

rhs(s) = min [c(s, s′) + g(s′)] for s ≠ goal

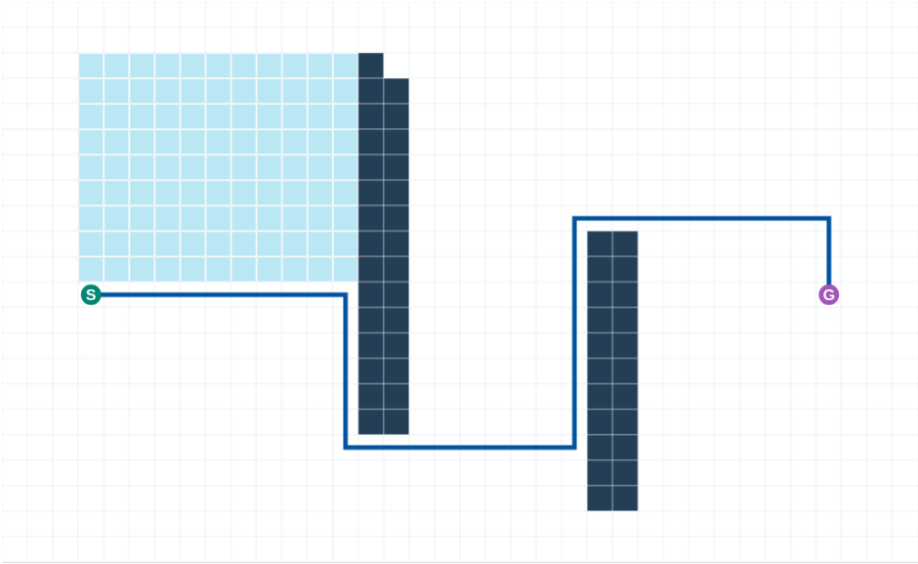
g(s) = rhs(s) means locally consistent

g is the stored estimate of cost to the goal.

rhs checks that estimate against one step through a neighbor.

A changed edge makes affected states inconsistent.

D* Lite repairs the route



The blocked cell changes the route.

Cyan shows the states processed by this repair.

The demo compares route cost and expansions with a fresh A* search.

Cyan: latest repair expansions · Blue: repaired route · Dark navy: blocked cells

Open interactive experiment

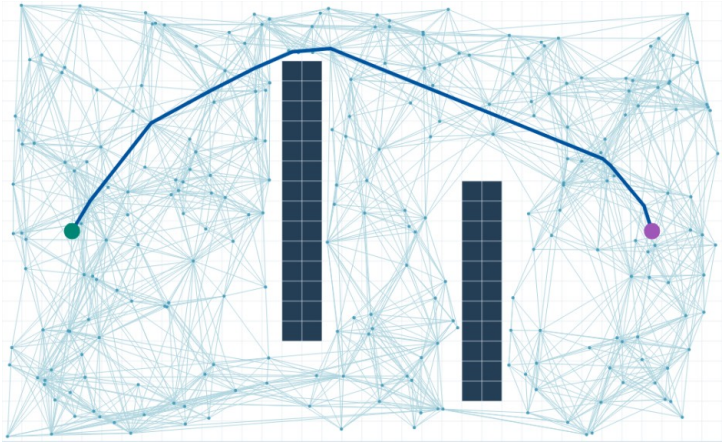
When incremental replanning helps

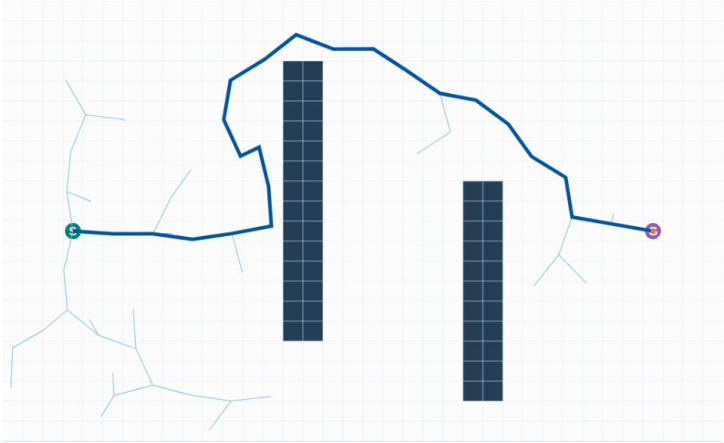
Situation	Expected benefit	Remaining problem
Few changed edges	Reuse unaffected estimates	Detect changes correctly
Robot advances	Retain goal-distance values	Maintain consistent priorities
Large map revisions	Benefit can shrink	Many states need repair
Moving traffic	Update current occupancy	Predict motion and timing

Popular planning families

Family	Examples	Useful problem
Graph search	Dijkstra, A*, D* Lite	Costed map / state graph
Sampling	PRM, RRT, RRT*	Continuous configuration space
Potential fields	Attraction / repulsion	Fast reactive motion guidance
Trajectory optimization	MPC, MPPI, trajectory fitting	Motion with costs and constraints

PRM and RRT





PRM: reusable roadmap

RRT: stop at the first route

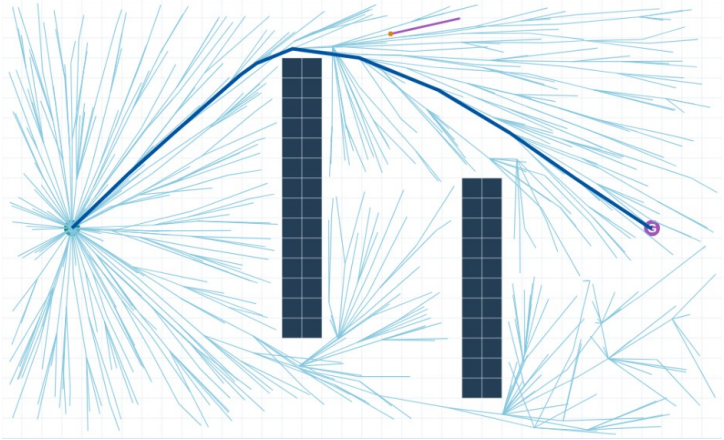
PRM connects free samples for repeated queries.
RRT extends a tree toward samples from one start.

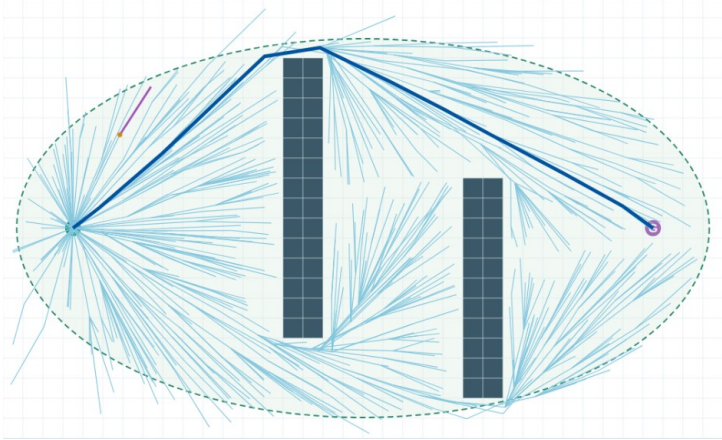
Pale cyan: connections · Blue: route · Orange: attempted RRT sample

Open PRM

Open RRT

RRT* and Informed RRT*





RRT*: rewire to improve cost

Informed RRT*: focus new samples

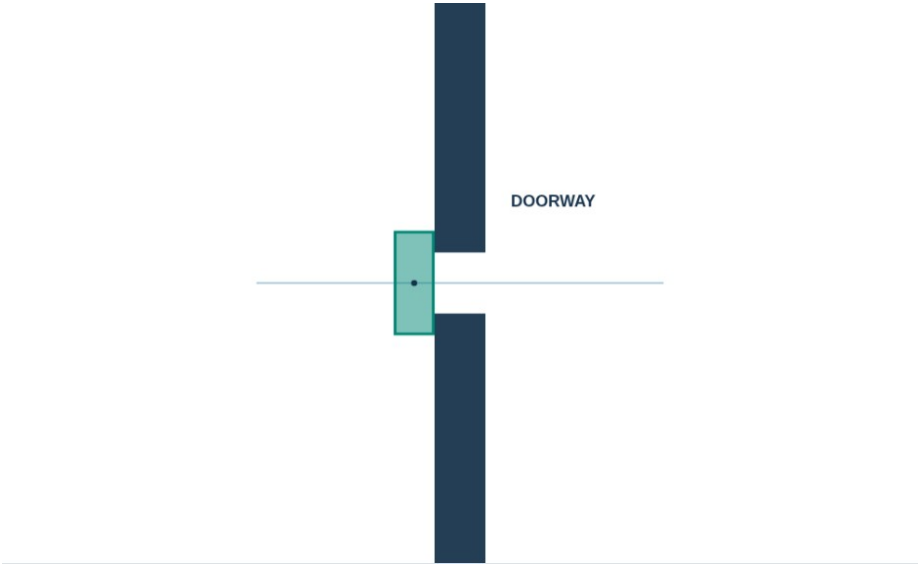
Both keep improving after a first route.
Informed RRT* then samples the green ellipse.

Cyan: tree · Magenta: latest rewires · Blue: best route
Dashed green: improvement ellipse · Orange: attempted sample

Open RRT*

Open Informed RRT*

The robot occupies space



A point path ignores the physical footprint.

Configuration includes every variable needed to locate the body.

A mobile rectangle needs position and orientation.

Green rectangle: last clear pose · Pale blue: center path · Dark navy: walls

Open interactive experiment

Obstacle expansion

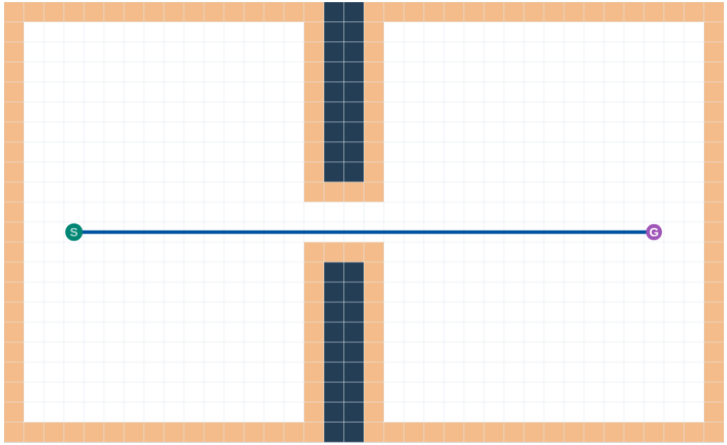
$C_{obstacle} = O \oplus (-R)$

Circular robot: expansion radius = r + margin

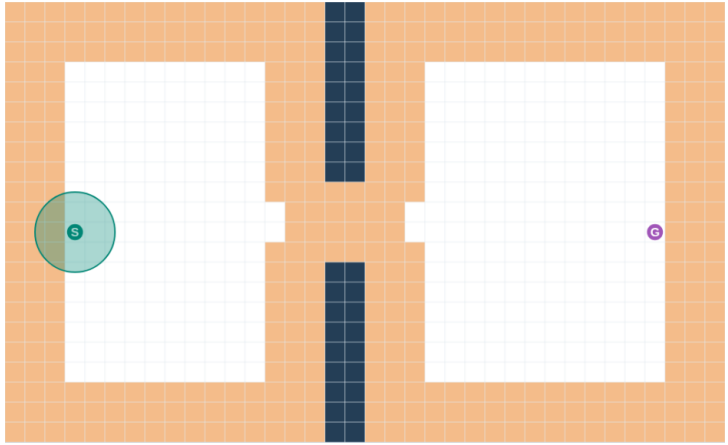
A circular robot of radius r expands each obstacle by r.

A safety margin adds clearance for model and pose error.

A passage closes as clearance grows



Small robot and margin



Larger robot and margin

A small footprint leaves a center route through the gap.
A larger footprint and margin remove that route.

Dark navy: obstacles · Orange: excluded cells · Blue: center route

Open interactive experiment

Hard clearance and soft cost

Mechanism	Effect	Tuning consequence
Footprint collision	Reject overlap with obstacles	Physical feasibility
Extra safety margin	Exclude additional space	Can close narrow gaps
Graded obstacle cost	Prefer farther clearance	Trades clearance against length
Excessive repulsion	Avoid high-cost regions	Can stall in narrow corridors

A vehicle cannot follow every point path

$q = (x, y, \theta)$

Turning radius $\approx L / \tan(\delta)$

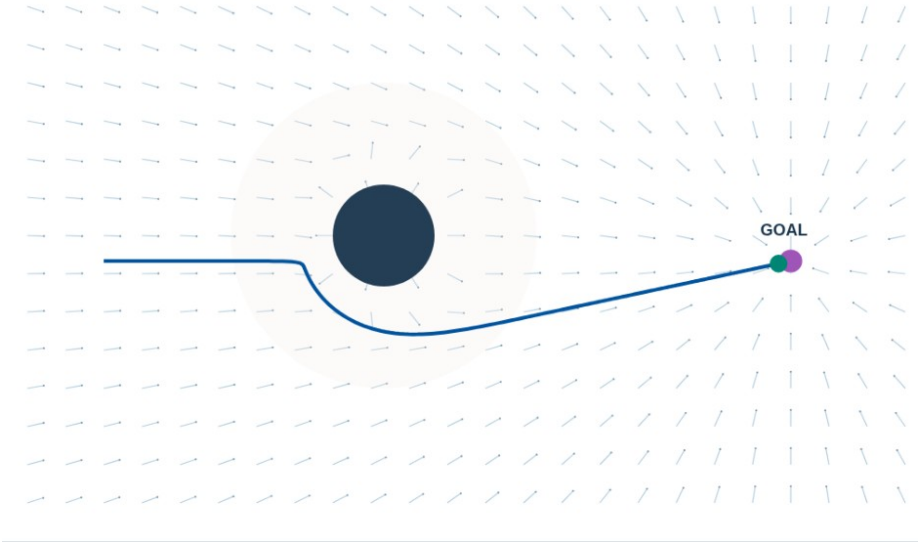
Steering constrains curvature.

Speed and braking limit what can happen next.

Hybrid A* and state lattices

Method	State / transitions	Why use it
Grid A*	Position and grid steps	Simple holonomic planning
Hybrid A*	Pose with curved motions	Respect turning constraints
State lattice	Pose and motion primitives	Search executable connections
Remaining check	Full footprint and timing	Steering feasibility alone is insufficient

Attraction and repulsion



Blue-gray strokes: net force · Blue: executed trace · Green: robot

Open interactive experiment

The goal supplies an attractive potential.

Nearby obstacles supply repulsive potentials.

The robot follows the negative gradient of their sum.

An artificial potential field

$U_{att} = \frac{1}{2} k \|p - goal\|^2$

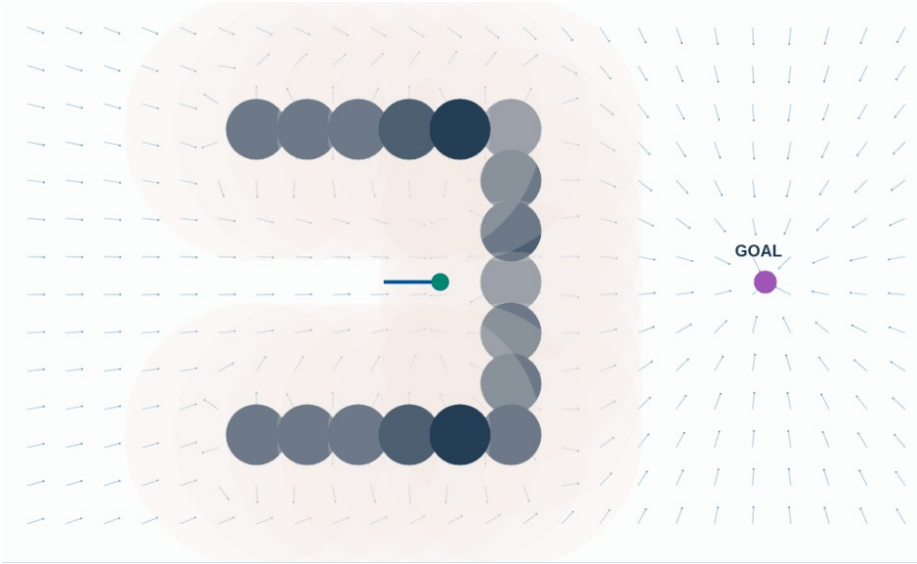
$U_{rep} = \frac{1}{2} \eta (1/d - 1/d_0)^2 \quad \text{for } d < d_0$

$\text{Motion direction} = -\nabla(U_{att} + \Sigma U_{rep})$

Attraction grows with distance from the goal.

Repulsion grows rapidly near an obstacle.

The U-shaped potential-field trap



Blue-gray strokes: net force · Blue: executed trace · Purple: goal

Open interactive experiment

Attraction points toward a goal beyond the wall.

Repulsion balances attraction inside the U.

The robot stops with the goal still far away.

Responses to potential-field failure

A global route can provide waypoints around the U.

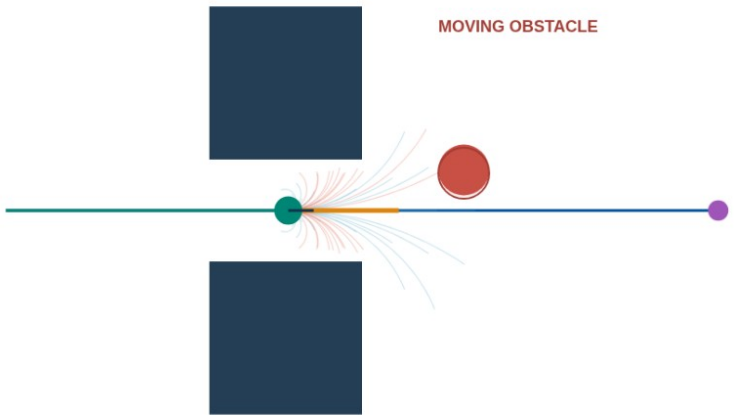
Recovery behaviors can leave a stalled configuration.

A longer-horizon search can evaluate a detour before moving.

The global and local navigation loop

1. The global planner computes a route to the goal.
2. The local planner predicts candidate motions near the robot.
3. A controller executes a short part and receives a new state.
4. A blocked route or missing progress triggers replanning or recovery.

Local trajectory sampling



Each candidate uses a different forward speed and turn rate.

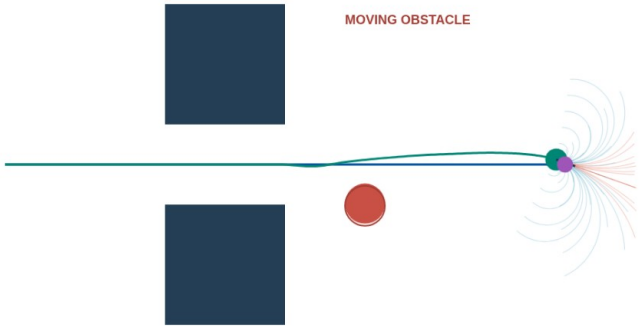
Sampled collision checks reject predicted overlaps.

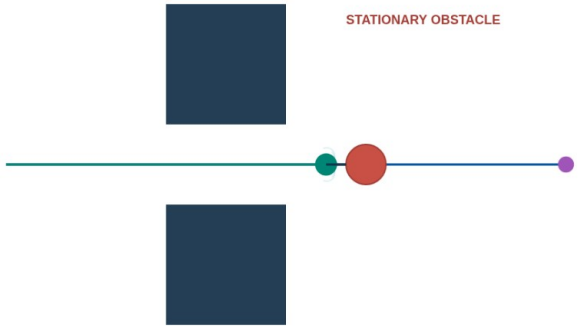
A score balances progress, path following, and motion.

Blue: global route · Cyan / pale red: accepted / rejected predictions
Orange: chosen prediction · Green: executed motion · Solid red: obstacle now

Open interactive experiment

A crossing obstacle and a permanent blockage





Moving obstacle clears the route

Stationary obstacle blocks the route

A moving obstacle clears and local execution progresses.

A permanent blockage stalls the controller until the route changes.

Blue: global route · Green: executed motion · Cyan / pale red: predictions

Solid red: obstacle now · Thin red outline: obstacle at planning time

Open interactive experiment

Common local planning approaches

Approach	Mechanism	Typical limitation
DWA / DWB	Score candidate velocity motions	Sampling and critic tuning
MPC	Optimize predicted controls	Model error and solve budget
MPPI	Improve sampled control sequences	Cost / noise / horizon tuning
Path following	Track a given reference	Avoidance depends on controller

Compare the MPPI-style controller (slide 44)

Prediction horizon and stopping distance

Distance ≈ v $t_{reaction}$ + v^2 / (2 a_{brake})

Horizon distance ≈ v × prediction time

A short horizon can hide a future collision.

A longer horizon costs more computation and needs useful predictions.

Planning in different environments

Environment	Planning emphasis	Common failure
Static warehouse	Map search and footprint	Narrow aisles / loaded robot
People in a corridor	Local prediction and stopping	Wrong motion forecast
Outdoor rough terrain	Traversability and replanning	Map misses slope / traction
Road vehicle	Feasible timed maneuvers	Grid corner / crossing actor

Greg Chirikjian’s Robot Imagination

Reconstruct an unfamiliar object for simulation.

Test candidate physical interactions and evaluate their outcomes.

Use the resulting affordance to guide a real action.

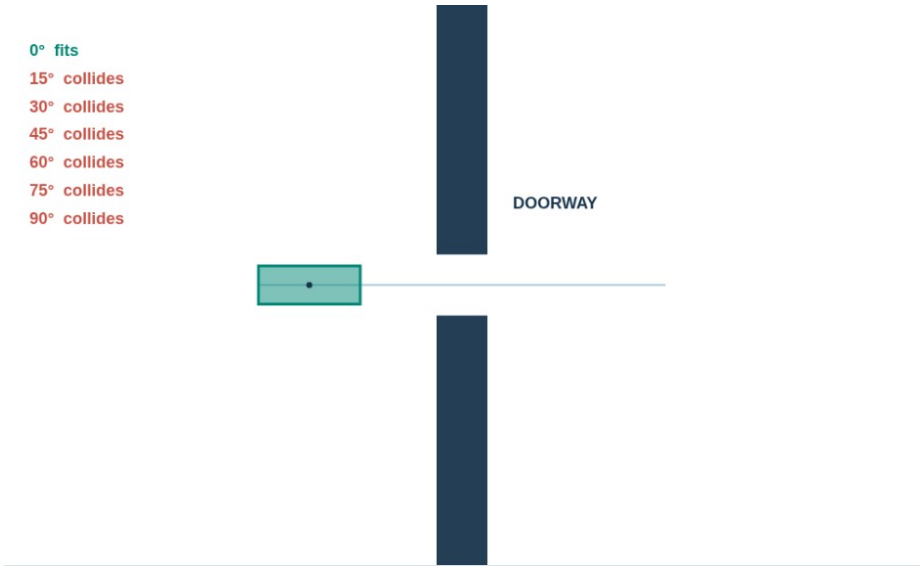
Imagination and motion planning

PRIMP learns distributions of demonstrated end-effector motions.

Planning uses those distributions to adapt motions around new obstacles.

RAIL combines language-based reasoning with simulated affordance tests.

Demo: imagined doorway motions



The center follows the same straight route.

Changing orientation changes the swept robot footprint.

Candidate motions must pass collision checks before execution.

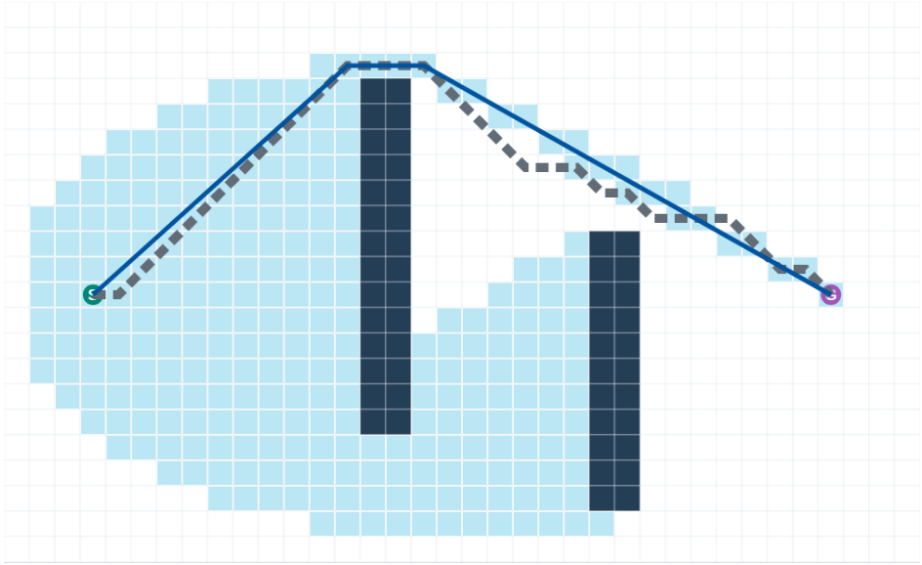
Green / red labels: candidate fits / collides · Green rectangle: clear pose

Open interactive experiment

Choosing a planning approach

Problem	Candidate approach	First check
Known cost map	A* / Dijkstra	Heuristic and graph costs
Changing map, same goal	D* Lite	Cost-change updates
Arm / complex geometry	PRM / RRT / RRT*	Configuration collision model
Vehicle maneuvers	Hybrid A* / lattice	Curvature and footprint
Moving nearby objects	Local prediction / optimization	Timing and stopping feasibility

Theta* and the A* reference



Dashed gray: eight-neighbor A* reference · Blue: Theta* route
Cyan: expanded cells · Dark navy: obstacles

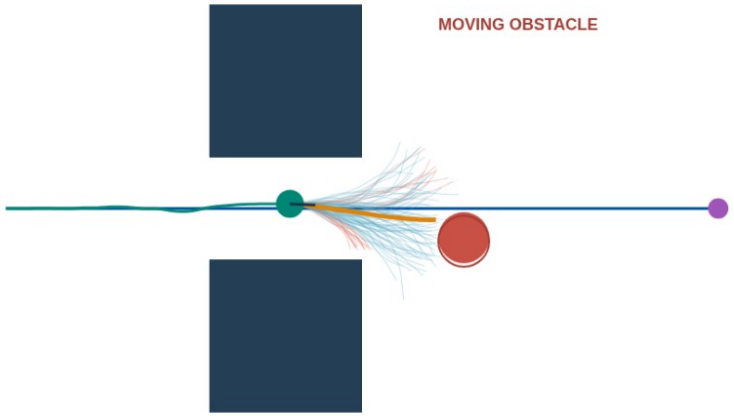
Open interactive experiment

Eight-neighbor A* follows grid directions.

Theta* tests direct connections through a state's parent.

A shorter geometric route may still violate vehicle steering limits.

MPPI-style local sequence optimization



Perturb a sequence of speeds and turn rates.

Combine controls using exponential cost weights.

Recheck the proposal, execute 0.1 s, and repeat.

Cyan / pale red: accepted / rejected predictions · Orange: chosen prediction
Blue: global route · Green: executed motion · Solid red: obstacle now

Open interactive experiment

Interactive experiments

1. Search: compare BFS, Dijkstra and A* across terrain. Record path cost and expansions.
2. Replanning: block and reopen a route cell. Compare D* Lite with fresh A*.
3. Geometry: grow the footprint, then rotate a rectangle at the doorway.
4. Local motion: change horizon and obstacle motion. Explain rejected candidates.

Open interactive experiment

Discussion and exit questions

Why can an optimal A* path still be unsafe for a vehicle?

When might D* Lite repair cost more work than a fresh search?

Why can a local planner stop while a global route still exists?

Which assumptions would an imagination model need before real execution?

Reading and resources

LaValle, Planning Algorithms: search, configuration space, sampling, feedback.

Stentz: D*. Koenig and Likhachev: D* Lite.

Nav2 documentation: planners, controllers, footprints, and cost tuning.

Chirikjian and collaborators: Robot Imagination, PRIMP, and RAIL.

Open the nine interactive experiment families